

AIM

Create a Cryptocurrency using Python and perform mining in the Blockchain created.

THEORY

- **Blockchain Structure:** A blockchain is an immutable, distributed digital ledger organized into chronologically linked blocks. Each block stores its index, a generation timestamp, a cryptographic Proof-of-Work (PoW), transaction data, and the SHA-256 cryptographic hash of the preceding block.
- **Mempool and Transactions:** Unconfirmed transactions are initially held in a memory pool (mempool) via the self.transactions list. When a miner creates a block, these pending transactions are collected, a miner reward transaction is credited to incentivize computation, and the pending pool is emptied (self.transactions = []) to prevent double-spending.
- **Proof-of-Work (PoW):** Miners solve a computationally intensive puzzle where a hash operation on the nonces/proofs must satisfy a target difficulty (e.g., yielding four leading zeros: hash_operation[:4] == '0000').
- **Consensus Protocol (Longest Chain Rule):** In a decentralized peer-to-peer network, consensus ensures all distributed ledgers stay synchronized. If multiple conflicting chains exist, nodes verify peer chains using cryptographic checks (is_chain_valid) and adopt the valid chain with the greatest length via replace_chain().

STEPS TO PERFORM EXPERIMENT

- **Step 1: Environment Setup & Node Launch**
 - Open three independent terminal tabs in your workspace.
 - Start the three decentralized nodes using Python:
 - Terminal 1: python3 hadcoin_node_5001.py
 - Terminal 2: python3 hadcoin_node_5002.py
 - Terminal 3: python3 hadcoin_node_5003.py
- **Step 2: Connect Peer Network**
 - Interconnect all three nodes by sending POST requests to /connect_node with the peer URLs in the JSON body:
 - Node 5001: POST
http://127.0.0.1:5001/connect_node with Body
{
 "nodes": [
 "http://127.0.0.1:5002",
 "http://127.0.0.1:5003"
]
}
 - Node 5002: POST
http://127.0.0.1:5002/connect_node with Body
{
 "nodes": [
 "http://127.0.0.1:5001",
 "http://127.0.0.1:5003"
]
}
 - Node 5003: POST
http://127.0.0.1:5003/connect_node with Body

```
{"nodes": [{"http://127.0.0.1:5001"}(http://127.0.0.1:5001)",
["http://127.0.0.1:5002"}(http://127.0.0.1:5002)"]}]}
```

- **Step 3: Add Transactions to Node 5001**

- Submit three separate POST requests to http://127.0.0.1:5001/add_transaction:
 - Request 1: {"sender": "a", "receiver": "b", "amount": 51}
 - Request 2: {"sender": "a", "receiver": "b", "amount": 52}
 - Request 3: {"sender": "a", "receiver": "b", "amount": 53}

- **Step 4: Mine Block on Node 5001**

- Trigger Proof-of-Work by sending a GET request to http://127.0.0.1:5001/mine_block.
- Confirm that Block 2 is created containing the 3 user transactions plus 1 miner reward transaction.

- **Step 5: Verify Asymmetrical Chain State**

- Inspect the chain lengths across nodes:
 - GET http://127.0.0.1:5001/get_chain \rightarrow Length: 2
 - GET http://127.0.0.1:5002/get_chain \rightarrow Length: 1
 - GET http://127.0.0.1:5003/get_chain \rightarrow Length: 1

- **Step 6: Consensus & Network Synchronization**

- Update the lagging nodes to resolve chain conflict using the Longest Chain Rule:
 - Run GET http://127.0.0.1:5002/replace_chain
 - Run GET http://127.0.0.1:5003/replace_chain
- Run GET /get_chain across all three nodes to verify ledger consistency (all showing length 2).

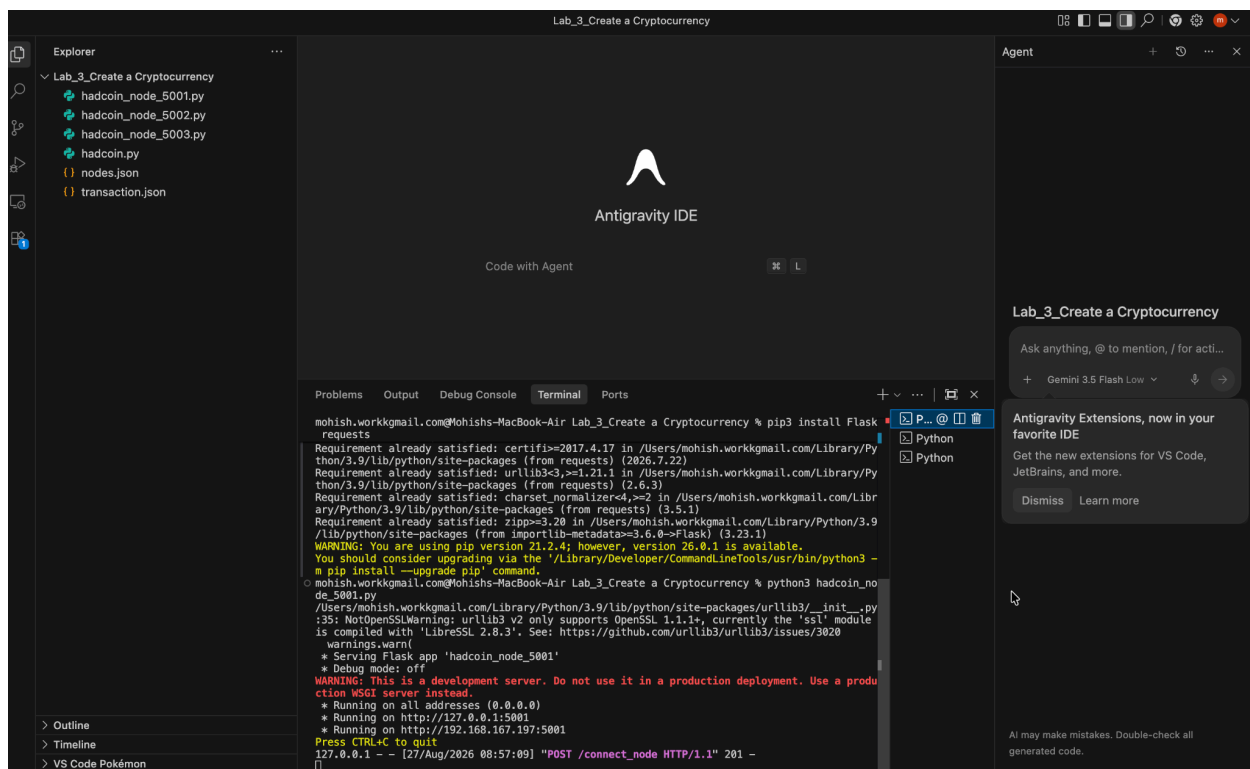
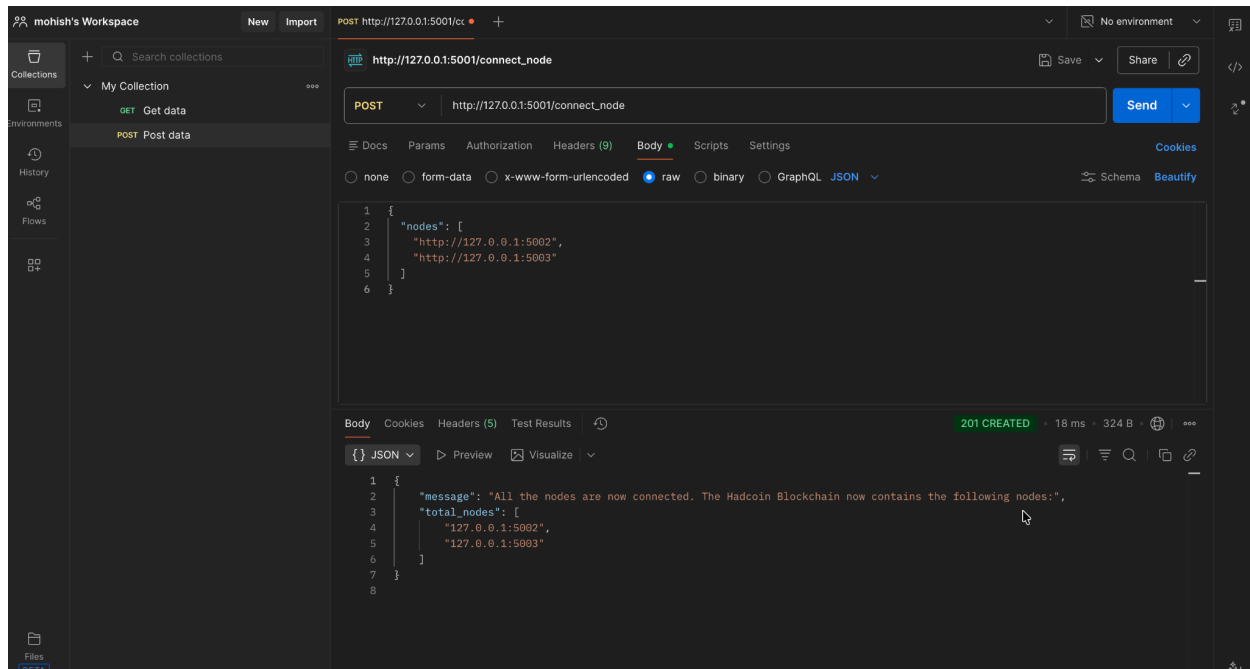
SCREENSHOTS

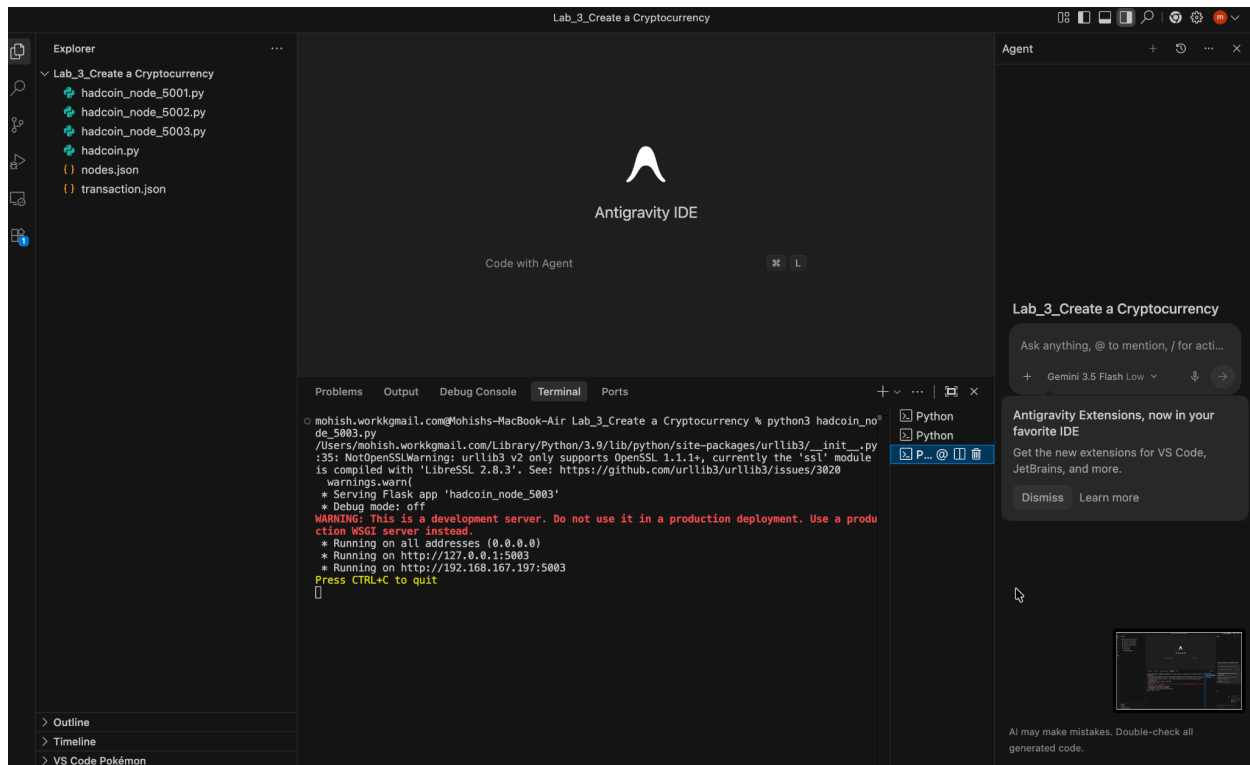
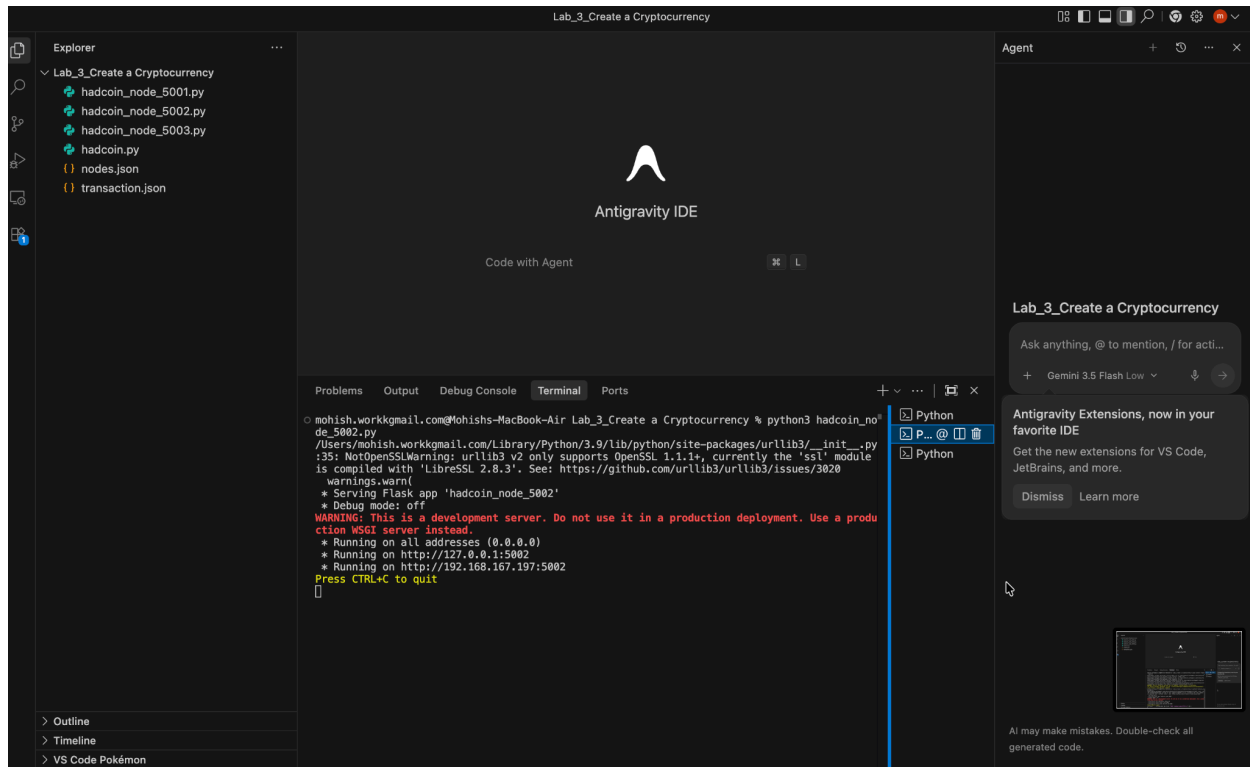
(Paste your output screenshots here in the following order:)

1. **Terminal Execution:** 3 terminal windows running nodes 5001, 5002, and 5003.
2. **Peer Connection:** Postman POST /connect_node responses for ports 5001, 5002, and 5003.
3. **Transaction Creation:** Postman POST /add_transaction responses (Blocks 51, 52, 53).
4. **Mining Execution:** Postman GET /mine_block response from Node 5001.
5. **Initial Chain Check:** Postman GET /get_chain for Node 5001 (Length: 2) and Node 5002 (Length: 1) showing length discrepancy.
6. **Chain Replacement (Consensus):** Postman GET /replace_chain responses for Node 5002 and Node 5003.

7. Final Verification: Postman GET /get_chain responses showing all nodes synchronized with Length 2.

SS:





Postman interface showing a successful POST request to `http://127.0.0.1:5002/connect_node`. The request body is a JSON object with a `nodes` array containing two URLs: `http://127.0.0.1:5001` and `http://127.0.0.1:5003`. The response is a `201 CREATED` status with a JSON body containing a `message` and a `total_nodes` array.

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body: `201 CREATED` · 13 ms · 324 B

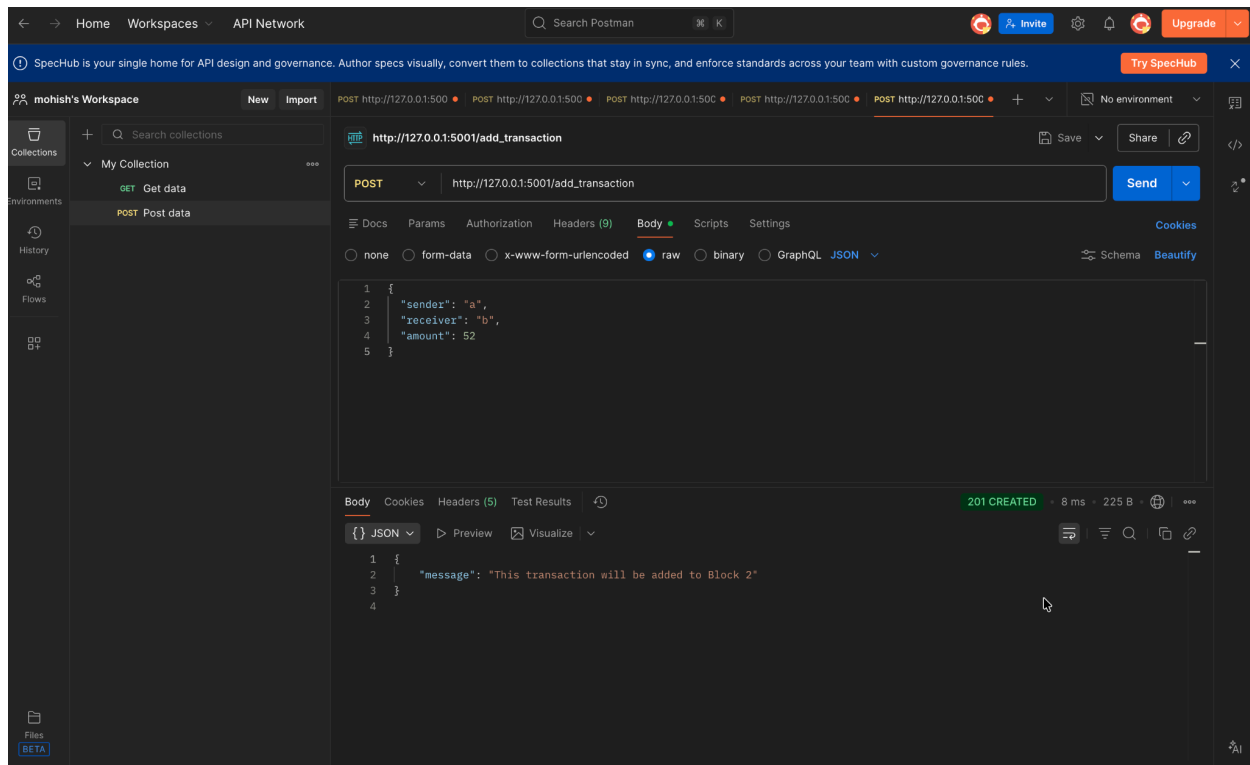
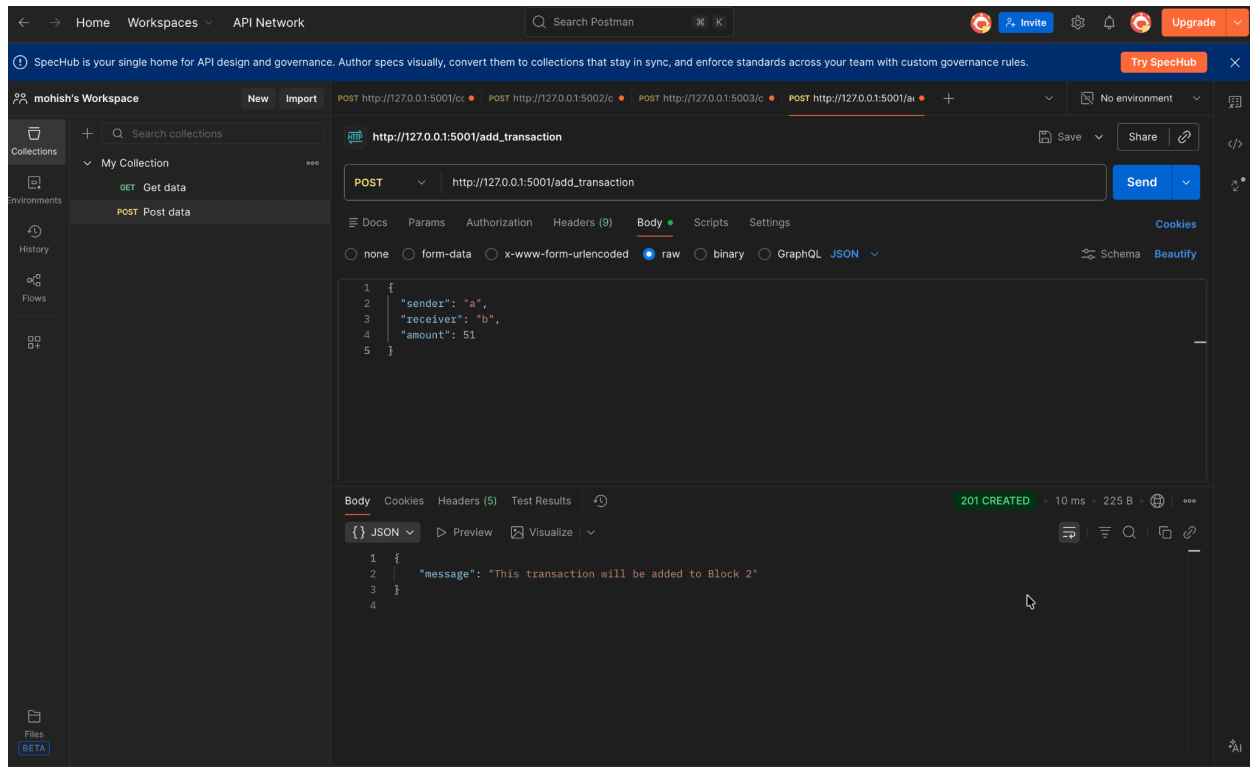
```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5003",
5     "127.0.0.1:5001"
6   ]
7 }
8
```

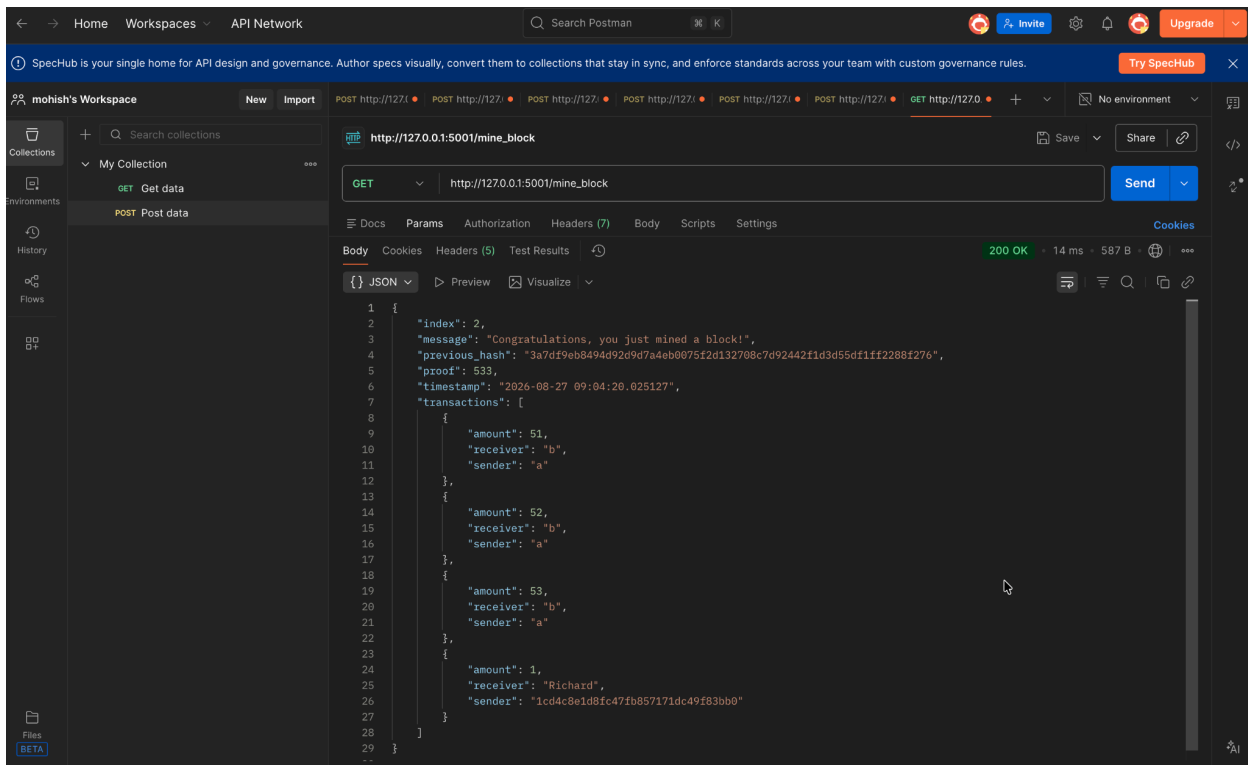
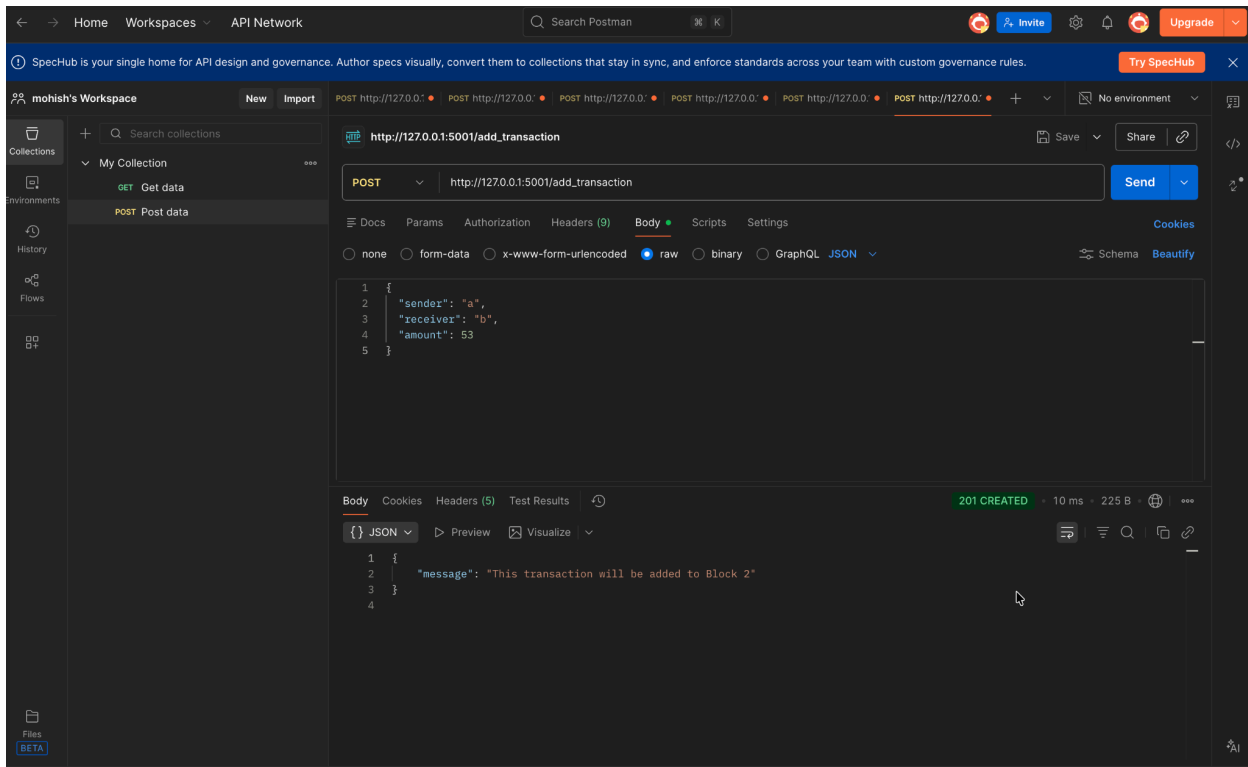
Postman interface showing a successful POST request to `http://127.0.0.1:5003/connect_node`. The request body is a JSON object with a `nodes` array containing two URLs: `http://127.0.0.1:5001` and `http://127.0.0.1:5002`. The response is a `201 CREATED` status with a JSON body containing a `message` and a `total_nodes` array.

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

Body: `201 CREATED` · 15 ms · 324 B

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5001"
6   ]
7 }
8
```





Postman interface showing a successful GET request to `http://127.0.0.1:5002/replace_chain`. The response is a JSON object with a chain of transactions.

```
11 {
12   "index": 2,
13   "previous_hash": "3a7df9eb8494d92d9d7a4eb0975f2d132708c7d92442f1d3d55df1ff2288f276",
14   "proof": 533,
15   "timestamp": "2026-08-27 09:04:20.025127",
16   "transactions": [
17     {
18       "amount": 51,
19       "receiver": "b",
20       "sender": "a"
21     },
22     {
23       "amount": 52,
24       "receiver": "b",
25       "sender": "a"
26     },
27     {
28       "amount": 53,
29       "receiver": "b",
30       "sender": "a"
31     },
32     {
33       "amount": 1,
34       "receiver": "Richard",
35       "sender": "1cd4c8e1d8fc47fb857171dc49f83bb0"
36     }
37   ]
38 }
39 ]
```

Postman interface showing a successful GET request to `http://127.0.0.1:5003/replace_chain`. The response is a JSON object with a message and a new chain of transactions.

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-08-27 08:54:37.237602",
9       "transactions": []
10    },
11    {
12       "index": 2,
13       "previous_hash": "3a7df9eb8494d92d9d7a4eb0975f2d132708c7d92442f1d3d55df1ff2288f276",
14       "proof": 533,
15       "timestamp": "2026-08-27 09:04:20.025127",
16       "transactions": [
17         {
18           "amount": 51,
19           "receiver": "b",
20           "sender": "a"
21         },
22         {
23           "amount": 52,
24           "receiver": "b",
25           "sender": "a"
26         },
27         {
28           "amount": 53,
29           "receiver": "b",
30           "sender": "a"
31         }
32       ]
33     }
34   ]
35 }
```